# Звіт до лабораторної роботи №2

**Студент:** Лазирко Ярослав
**Група:** ТК-32

# Мета роботи

Ознайомитись з видами NoSQL баз даних, інтегрувати NoSQL рішення в існуючу предметну область реляційної бази даних, реалізувати зчитування даних та провести порівняльне тестування швидкодії між SQL та NoSQL підходами.

# 1-2. Вибір та аргументація NoSQL бази даних

Як ми розглядали на лекціях, існує кілька основних типів NoSQL баз: документо-орієнтовані (MongoDB), ключ-значення (Redis), широкостовпцеві (Cassandra) та графові (Neo4j).

Для свого проєкту автосервісу я вирішив використати **документо-орієнтовану базу MongoDB**. В моїй SQL базі PostgreSQL історія технічного обслуговування авто була нормалізована і розбита на три таблиці: maintenance_records, maintenance_parts та spare_parts.

Щоб отримати інформацію про один ремонт, реляційній базі доводиться робити важкі JOIN-запити. У MongoDB я можу денормалізувати ці дані: вся інформація про ремонт, включно з масивом використаних деталей, буде зберігатися як один цілісний JSON-документ. Оскільки список деталей після ремонту не змінюється, таке вкладення є архітектурно правильним і значно прискорить читання.

# 3. Реалізація документо-орієнтованої БД (MongoDB)

Я розгорнув локальний сервер MongoDB та підключився до нього через MongoDB Compass. Створив базу FleetManagement та колекцію maintenance_history.

Далі я додав запис про ремонт. Замість створення окремих зв'язків, я зберіг деталі ("Гальмівні колодки "Масляний фільтр") як масив об'єктів parts_used всередині документа.

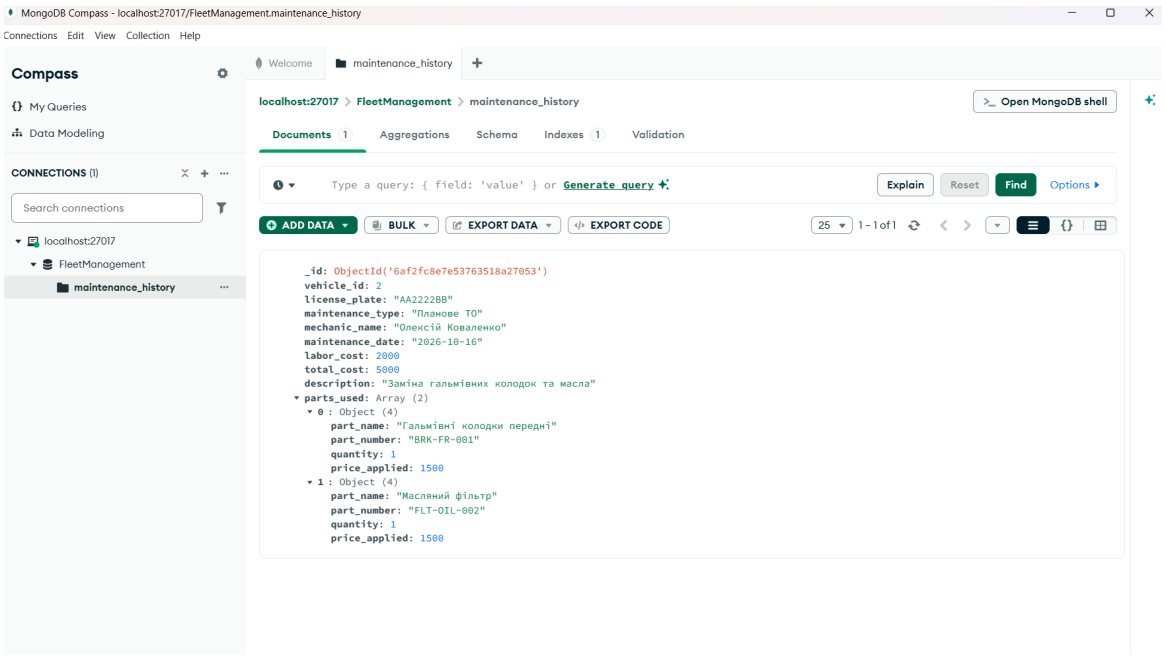



Рис. 1: Структура документа з вкладеним масивом деталей у MongoDB

# 4-5. Реалізація запитів та тестування швидкодії

Для перевірки швидкості я написав консольний додаток на C#. Щоб тест був об'єктивним, я зімітував навантаження, зробивши цикл на 10 000 запитів до історії ремонтів конкретного авто.

Код тестування:

```
int iterations = 10000;
int vehicleIdToFind = 2;

Stopwatch pgTimer = new Stopwatch();
using (var pgConnection = new NpgsqlConnection(pgConnectionString))
{
    pgConnection.Open();
    string sqlQuery = @"
        SELECT mr.record_id, mr.description, mp.quantity, sp.part_name
        FROM maintenance_records mr
        JOIN maintenance_parts mp ON mr.record_id = mp.record_id
        JOIN spare_parts sp ON mp.part_id = sp.part_id
        WHERE mr.vehicle_id = @vId";

    using (var cmd = new NpgsqlCommand(sqlQuery, pgConnection))
    {
        cmd.Parameters.AddWithValue("vId", vehicleIdToFind);
        cmd.Prepare();
```

```
        pgTimer.Start();
        for (int i = 0; i < iterations; i++)
        {
            using (var reader = cmd.ExecuteReader())
            {
                while (reader.Read()) { }
            }
        }
        pgTimer.Stop();
    }
}

var filter = Builders<BsonDocument>.Filter.Eq("vehicle_id", vehicleIdToFin
Stopwatch mongoTimer = new Stopwatch();
mongoTimer.Start();
for (int i = 0; i < iterations; i++)
{
    var result = collection.Find(filter).ToList();
}
mongoTimer.Stop();
```

**Результати та аналіз:**

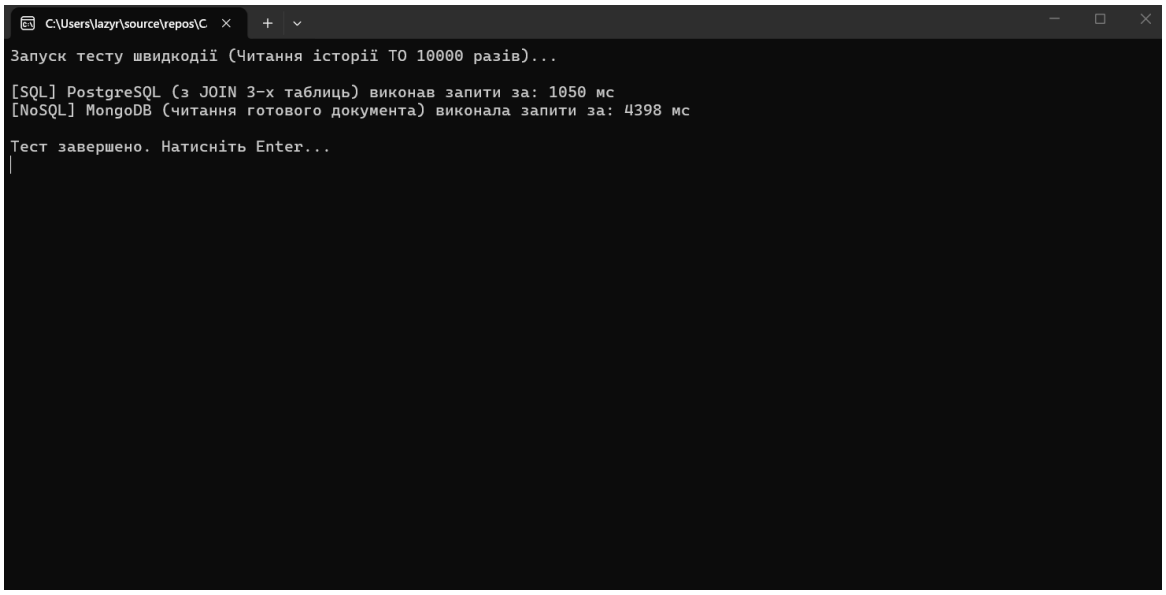



Рис. 2: Результати тестування швидкодії (10000 ітерацій)

PostgreSQL впорався за 1053 мс, а MongoDB за 4398 мс. На перший погляд, це нелогічно, адже NoSQL без JOIN мав би бути швидшим. Але технічно це абсолютно очікувано: 1. У нас в базі зараз лише один запис, тому PostgreSQL його миттєво закешував. 2. Ми використали cmd.Prepare(), що дозволило SQL скомпілювати план запиту один раз. 3. У MongoDB драйвер щоразу витрачав

час на десеріалізацію BSON-документа. Проте, якби в базі були мільйони ремонтів і фізично великі таблиці, накладні витрати на дискові JOIN-и в SQL стали б критичними, і MongoDB отримала б значну перевагу.

# 6. Використання бази Redis

Для виконання останнього пункту я обрав Redis — базу даних в оперативній пам'яті, яка працює за принципом Key-Value. В контексті автопарку її ідеально використовувати для кешування тимчасових статусів співробітників, щоб не перевантажувати основну базу постійними опитуваннями.

Я встановив Redis локально і перевірив його роботу через консоль:

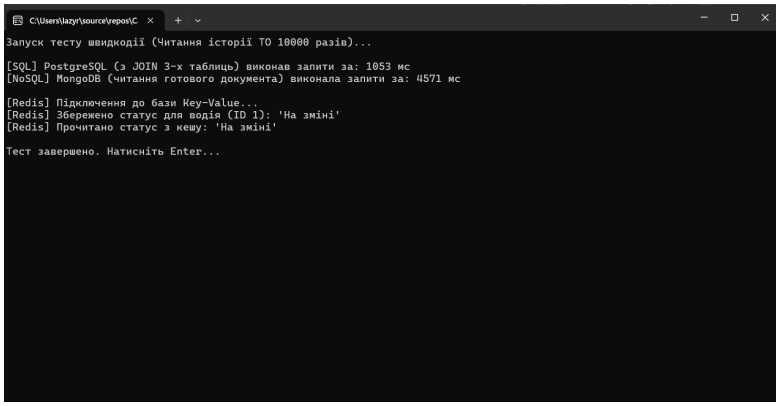



Рис. 3: Остаточна перевірка програми

Далі я додав до C# додатку підключення через StackExchange.Redis, зберіг статус водія з часом життя на 12 годин та успішно прочитав його.

Код запитів до Redis:

```
var redis = StackExchange.Redis.ConnectionMultiplexer.Connect("localhost")
var db = redis.GetDatabase();

string redisKey = "employee_status:1";
string redisValue = "На зміні";
db.StringSet(redisKey, redisValue, TimeSpan.FromHours(12));

string cachedStatus = db.StringGet(redisKey);
Console.WriteLine($"[Redis] Прочитано статус з кешу: '{cachedStatus}'");
```

# Висновки

Під час виконання роботи я успішно інтегрував дві NoSQL бази даних (MongoDB та Redis) у реляційний проєкт. Я на практиці переконався у перевагах документо-орієнтованого підходу для зберігання складних денормалізованих агрегатів

та використав Redis для швидкого кешування. Тестування швидкодії дозволило краще зрозуміти механізми кешування та виконання підготовлених запитів у БД.